

پروژه عملی درس نظریه زبان ها و ماشین ها

بازی زندگی کانوی و مورچه لنگتون

دکتر انتظاری - دکتر باغبانی

طراحان: میلاد زارعی، طاهای بیکلریان

ترم ۴۰۴۲



دانشگاه علم و صنعت ایران

فهرست مطالب

۲	پروژه اتوماتای سلولی: بازی زندگی کانوی و مورچه لنگتون
۲	۱. مقدمه و مبانی نظری اتوماتای سلولی
۳	۲. بخش اول - بازی زندگی کانوی (Game of Life)
۳	قوانین چهارگانه بازی زندگی (GoL Rules)
۳	تکالیف بخش اول:
۴	۳. بخش دوم - مورچه لنگتون (Langton's Ant)
۴	قوانین مورچه لنگتون (Ant Rules)
۵	تکالیف بخش دوم:
۵	۴. بخش سوم - جامعیت تورینگ از طریق گیت‌های منطقی (Turing Completeness)
۶	تکالیف بخش سوم:
۶	لینک‌های کمکی
۶	لینک‌های جالب
۷	مراجع

پروژه اتوماتای سلولی: بازی زندگی کانوی و مورچه لنگتون

اتوماتای سلولی (Cellular Automata) مدل‌های ریاضی گسسته و بسیار قدرتمندی هستند که در نظریه محاسبات و فیزیک نظری کاربردهای فراوانی دارند. این سیستم‌ها از یک شبکه متناهی یا نامتناهی از سلول‌ها تشکیل شده‌اند که هر کدام در یکی از حالات متناهی قرار دارند. حالت بعدی هر سلول تنها بر اساس حالت فعلی آن و همسایگان مجاورش مشخص می‌شود.

یکی از شگفت‌انگیزترین نتایج اتوماتای سلولی، پدیده «نوظهوری» (emergence) است؛ جایی که از قوانین محلی بسیار ساده، رفتارهای سراسری فوق‌العاده پیچیده و حتی «جامعیت تورینگ» (Turing Completeness) پدیدار می‌شود. این به آن معناست که برخی سیستم‌های ساده اتوماتا قادرند هر نوع محاسبه ریاضی یا الگوریتم کامپیوتری ممکن را شبیه‌سازی نمایند.

در این پروژه عملی، شما با دو اتوماتای سلولی معروف یعنی **بازی زندگی کانوی** (Conway's Game of Life) و **مورچه لنگتون** (Langton's Ant) کار خواهید کرد. شما ابتدا شبیه‌سازهای پایه این اتوماتا را پیاده‌سازی کرده و در نهایت با طراحی گیت‌های منطقی دیجیتال از طریق جریان گلایدرها، جامعیت محاسباتی این جهان‌ها را اثبات خواهید نمود.

۱. مقدمه و مبانی نظری اتوماتای سلولی

در نظریه ماشین‌ها و زبان‌ها، هر سلول در یک اتوماتای سلولی را می‌توان به عنوان یک **ماشین حالت متناهی** (Finite State Machine - FSM) در نظر گرفت. فرض کنید سلول c در موقعیت (i, j) در یک شبکه دوبعدی $N \times N$ قرار دارد. حالت این سلول در زمان t با $s_t(c) \in \{0, 1\}$ نشان داده می‌شود که در آن 0 به معنای سلول مرده (dead) و 1 به معنای سلول زنده (alive) است.

تابع انتقال حالت اتوماتا را می‌توان به صورت زیر تعریف کرد:

$$\delta : S \times S^k \rightarrow S$$

که در آن $S = \{0, 1\}$ مجموعه حالات سلول، k تعداد همسایگان مجاور (مثلاً در همسایگی ۸-اتصال مور، $k = 8$) و δ تابع انتقال حالت است که حالت بعدی سلول را بر اساس حالت فعلی خودش و وضعیت زنده/مرده بودن همسایگانش در گام زمانی قبلی مشخص می‌کند:

$$s_{t+1}(c) = \delta \left(s_t(c), \sum_{n \in \text{Neigh}(c)} s_t(n) \right)$$

جهان محاسباتی اتوماتا به صورت گام‌به‌گام (tick) تکامل می‌یابد و تمام سلول‌ها به طور همزمان حالت خود را به‌روزرسانی می‌کنند.

۲. بخش اول – بازی زندگی کانوی (Game of Life)

بازی زندگی کانوی یک اتوماتای سلولی دوبعدی است که بر روی خانه‌های یک شبکه $N \times N$ اجرا می‌شود. تکامل هر سلول بر اساس همسایگی ۸-اتصال (Moore Neighborhood) بررسی می‌شود. قوانین انتقال بازی زندگی کانوی بسیار ساده و در قالب ۴ اصل زیر تعریف می‌شوند:

قوانین چهارگانه بازی زندگی (GoL Rules)

۱. **کم جمعیتی (Underpopulation):** هر سلول زنده با کمتر از ۲ همسایه زنده، در گام بعدی می‌میرد (تغییر حالت از ۱ به ۰).
 ۲. **بقا (Survival):** هر سلول زنده با ۲ یا ۳ همسایه زنده، در گام بعدی زنده می‌ماند (حفظ حالت ۱).
 ۳. **بیش جمعیتی (Overpopulation):** هر سلول زنده با بیش از ۳ همسایه زنده، در گام بعدی می‌میرد (تغییر حالت از ۱ به ۰).
 ۴. **تولیدمثل (Reproduction):** هر سلول مرده با دقیقاً ۳ همسایه زنده، در گام بعدی زنده و متولد می‌شود (تغییر حالت از ۰ به ۱).
- نکته:** اسکریپت های `conway.py`، `test_gameoflife_glider_simple.py` و `pygame_gol.py` را ببینید.

تکالیف بخش اول:

الف) پیاده‌سازی قوانین اصلی (TODO [Part 1a - Core Rules]) در فایل `con-way.py` و در متد `evolve` بخش مربوط به تکامل سنتی یا سلول به سلول را پیاده‌سازی کنید. دانشجو باید قوانین چهارگانه فوق را با پیمایش حلقه‌ای روی سطرها و ستون‌ها پیاده کند. حتماً به متغیر `self.finite` دقت کنید؛ اگر این متغیر `False` باشد، شبکه باید به صورت حلقوی یا توروئیدال (`wrapping boundaries`) رفتار کند؛ یعنی سطرها و ستون‌های مرزی به سطرها و ستون‌های سمت مقابل متصل شوند. صحت الگوریتم خود را با الگوی نوسان‌گر چشمک‌زن (Blinker) بسنجید.

ب) شبیه‌سازی گلايدر (TODO [Part 1b]) شبیه‌ساز را با ساختار گلايدر (Glider) اجرا کنید. این ساختار یک سفینه کوچک است که با سرعت مشخصی ($c/4$) به طور مورب در جهان محاسباتی حرکت می‌کند. توضیح دهید که چگونه می‌توان از حرکت گلايدر برای ارسال سیگنال و اطلاعات در جهان اتوماتا استفاده کرد.

- برای فهم بیشتر در مورد گلايدر ها به لینک زیر مراجعه کنید:

Digital Logic Gates on Conway's Game of Life –

ج) اشکال‌زدایی تفنگ گلايدر گوپر (TODO [Part 1c - Glider Gun Fix]) تفنگ گلايدر گوپر (Gosper Glider Gun) ساختاری دوره‌ای (oscillator) است که به طور نامتناهی جریان‌هایی از گلايدها را تولید و شلیک می‌کند. متد `insertGliderGun` در قالب پایه پروژه قرار داده شده است اما مختصات آن دارای یک خطای جزئی (شفت نامتقارن بلوک چپ) است که مانع از کارکرد صحیح تفنگ می‌شود. وظیفه شما این است که کد را تحلیل و دیباگ کرده و مختصات صحیح تفنگ را اصلاح نمایید تا جریان نامتناهی شلیک گلايدها پدیدار شود.

برای مطالعه بیشتر در مورد گلايدر گان‌ها می‌توانید به لینکی که بالا داده شده است مراجعه کنید.

د) پیاده‌سازی پارسر فایل‌های الگو (TODO [Part 1d - RLE/Plaintext Parser]) برای بارگذاری الگوهای بزرگ و پیچیده (بزرگتر از 20×20) از وب‌سایت مرجع LifeWiki، یک پارسر بنویسید. متد `parse_pattern(filepath)` را به گونه‌ای پیاده‌سازی کنید که بتواند فایل‌های با فرمت `.cells` (Plaintext) و همچنین فایل‌های فشرده‌شده به روش Run Length Encoded (`.rle`) را به درستی تحلیل و بارگذاری کند.

برای دانستن بیشتر در مورد فرمت‌های RLE و Plaintext می‌توانید به لینک زیر مراجعه کنید:

• RLE format on LifeWiki

• Plaintext format on LifeWiki

ه) بهینه‌سازی محاسباتی با کانولوشن (TODO [Part 1e - Fast Convolution]) برای شبکه‌های بسیار بزرگ ($N > 1024$)، پیاده‌سازی دستی سلول‌به‌سلول بسیار کند و سنگین است. $O(N^2)$ یک متد پیشرفته در متد `update_grid_fast(grid)` بنویسید که با استفاده از ضرب کانولوشن دوبعدی از کتابخانه علمی `scipy.signal.convolve2d`، همسایگی‌ها را به صورت برداری و با سرعت بالا محاسبه کند. این متد باید هم حالت مرزی محدود (`finite=True`) و هم حالت حلقوی توروئیدال (`finite=False`) را پوشش دهد.

برای مطالعه بیشتر در مورد کانولوشن در بازی زندگی کانوی می‌توانید به لینک زیر مراجعه کنید:

• Convolution in Game of Life

۳. بخش دوم – مورچه لنگتون (Langton's Ant)

مورچه لنگتون یک اتوماتای سلولی بسیار جذاب است که نشان‌دهنده یک ماشین تورینگ دوبعدی (2D Turing Machine) با قوانین بسیار ساده اما رفتار آشوبناک و نوظهور فوق‌العاده پیچیده است. مورچه روی یک شبکه سطر و ستونی حرکت می‌کند. هر خانه شبکه دارای رنگ مشخصی است. قوانین ابتدایی حرکت مورچه با دو رنگ سفید و سیاه به شرح زیر است:

قوانین مورچه لنگتون (Ant Rules)

۱. روی خانه سفید: اگر مورچه روی خانه سفید باشد، رنگ آن خانه را تغییر داده (به سیاه)، ۹۰ درجه به سمت راست (R) می‌چرخد و یک خانه به جلو حرکت می‌کند.

۲. **روی خانه سیاه:** اگر مورچه روی خانه سیاه باشد، رنگ آن خانه را تغییر داده (به سفید)، ۹۰ درجه به سمت چپ (L) می‌چرخد و یک خانه به جلو حرکت می‌کند.

نکته: اسکریپت‌های `langton.py` را ببینید.

تکالیف بخش دوم:

الف) پیاده‌سازی پایه مورچه (TODO [Part 2 - Langton's Ant]) در فایل جدید `langton.py` کلاس `LangtonsAnt` را از ابتدا و مطابق ساختار داده‌شده پیاده‌سازی کنید. مورچه باید موقعیت جاری خود را به صورت سطر و ستون (r, c) و جهت مواجهه فعلی خود را بر اساس چهار جهت اصلی (شمال، شرق، جنوب، غرب - URDL) نگهداری کند.

ب) شبیه‌سازی و اثبات سناریوها مورچه خود را اجرا کنید و نشان دهید که رفتارهای زیر را به درستی شبیه‌سازی می‌کند:

۱. **رفتار آشوبناک:** (Chaotic behaviour) در ابتدا مورچه برای هزاران گام به طور کاملاً بی‌نظم و آشوبناک حرکت کرده و الگوهای متقارن نامنظمی روی صفحه ایجاد می‌کند.
۲. **بزرگراه مورچه:** (The Highway) مورچه پس از حدود ۱۰,۰۰۰ گام، ناگهان وارد یک الگوی تکرارشونده و منظم ۱۰۴ گامی می‌شود که در قالب یک بزرگراه مورب، به طور نامتناهی پیش‌روی می‌کند.

ج) بهبود مورچه با پشتیبانی از چند رنگ (Multi-color States) پیاده‌سازی خود را تعمیم دهید تا از حالت‌های چندرنگی پشتیبانی کند. قوانین انتقال چندرنگی به صورت یک دیکشنری انتقال داده می‌شود. به عنوان مثال، در قوانین `LLRR` یا `RLR` رنگ هر خانه به رنگ بعدی در یک چرخه تغییر می‌کند و جهت چرخش مورچه بر اساس جدول انتقال مشخص می‌شود. نشان دهید مورچه شما می‌تواند الگوهای پیچیده و زیبایی نظیر مثلث متقارن یا الگوهای تکراری جالب را با قواعد چندرنگی رسم کند.

۴. بخش سوم - جامعیت تورینگ از طریق گیت‌های منطقی (Turing Completeness)

یکی از هیجان‌انگیزترین ویژگی‌های بازی زندگی کانوی، قابلیت انجام محاسبات عمومی است. هر مدار دیجیتال یا برنامه کامپیوتری را می‌توان در بستر جهان بازی زندگی شبیه‌سازی کرد. برای اثبات ریاضی جامعیت تورینگ، کافی است نشان دهیم که می‌توان با اتوماتا گیت‌های منطقی اصلی نظیر `AND` و `NOT` را پیاده‌سازی کرد.

در این بخش پروژه، سیگنال‌ها (بیت‌های ورودی) با استفاده از **گلایدرها** شبیه‌سازی می‌شوند:

- حضور یک گلايدر در یک بازه زمانی خاص معادل بیت یک (1) است.
- عدم حضور گلايدر در آن بازه معادل بیت صفر (0) است.

نکته: اسکریپت‌های `logic_gates.py` را ببینید.

تکالیف بخش سوم:

الف) پیاده‌سازی گیت (AND (TODO [Extension - Logic Gates] در فایل `logic_gates.py` متد `setup_and_gate` و `run_and_gate` را پیاده‌سازی نمایید. شما باید دو گلايدر ورودی (Glider A) و (Glider B) را در موقعیت‌هایی قرار دهید که مسیر حرکت آن‌ها با زاویه ۹۰ درجه تلاقی کند:

- **ورودی (۰،۰):** هیچ گلايدر شلیک نمی‌شود. خروجی در مختصات هدف (12, 15) صفر است.
- **ورودی (۱،۰) یا (۰،۱):** تنها یک گلايدر حرکت می‌کند، از مختصات تصادم عبور کرده و توسط مرز یا نابودکننده محو می‌شود. خروجی صفر است.
- **ورودی (۱،۱):** هر دو گلايدر شلیک شده، دقیقاً در زمان مشخصی با یکدیگر تصادم می‌کنند و بر اثر واکنش تصادم، یک ساختار بلوک ثابت 2×2 در مختصات هدف (15, 12) پدید می‌آورند.

این رفتار دقیقاً جدول درستی گیت منطقی و (AND) را پیاده‌سازی می‌کند.

ب) پیاده‌سازی گیت (NOT (TODO [Extension - Logic Gates] متدهای `setup_not_gate` و `run_not_gate` را پیاده‌سازی کنید. یک گیت NOT به یک سیگنال کمکی یا کنترل ثابت نیاز دارد:

- یک گلايدر کنترل (Control Glider) همواره تولید و شلیک می‌شود تا به سمت منطقه خروجی حرکت کند.
- اگر سیگنال ورودی A غیرفعال (0) باشد، گلايدر کنترل بدون برخورد عبور کرده و به منطقه خروجی می‌رسد (خروجی 1).
- اگر سیگنال ورودی A فعال (1) باشد، گلايدر A دقیقاً به گونه‌ای شلیک می‌شود که در مسیر با گلايدر کنترل برخورد کرده و هر دو یکدیگر را به طور کامل نابود (annihilation) می‌کنند (خروجی 0).

این واکنش دقیقاً عملکرد گیت منطقی **نقیض (NOT)** را مدل‌سازی می‌کند.

لینک‌های کمکی

- LifeWiki - The Game of Life Resource
- Inventing Game of Life (John Conway) - Numberphile
- Let's BUILD a COMPUTER in CONWAY's GAME of LIFE

لینک های جالب

- Video of an 8-bit Programmable computer in GoL!

Various GoL and Langton's Ant videos on the course Computation •
YouTube Playlist

مراجع

- Moore, C., Mertens, S., 2011. *The Nature Of Computation*. Oxford University Press.
- Wikipedia. *Conway's Game of Life & Langton's Ant universal turing machine proofs*.